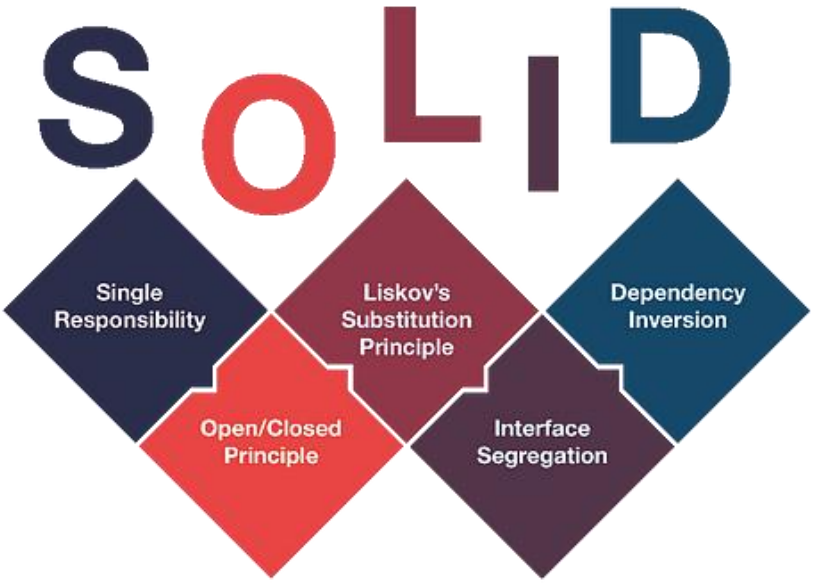
SOLID Design Principle Java



In this tutorial we will discuss about SOLID principle and its importance in Software development

Key contents

1. SOLID Introduction
2. What is SOLID Acronym

3. SOLID design principles

   https://medium.com/@javatechie/solid-design-principle-java-ae96a48db97

SOLID Introduction

In Java, **SOLID principles** are an object-oriented approach that are applied to software structure design. It is conceptualized by **Robert C. Martin** (also known as Uncle Bob). These five principles have changed the world of object-oriented programming, and also changed the way of writing software. It also ensures that the software is modular, easy to understand, debug, and refactor. In this section, we will discuss **SOLID principles in Java** with proper example**.**

SOLID Acronym

S : Single Responsibility Principle (SRP)

O : Open closed Principle (OSP)

L : Liskov substitution Principle (LSP)

I : Interface Segregation Principle (ISP)

D : Dependency Inversion Principle (DIP)

Now let's deep dive into what all SOLID principle we have with Example

SOLID design principles

Single Responsibility Principle

This principle states that "*a class should have only one reason to change*" which means every class should have a single responsibility or single job or single purpose

The principle can be well understood with an example. Imagine there is a class called ***BankService*** which performs following operations.

1. Withdraw
2. Deposit
3. Print Pass Book
4. Get Loan Info
5. Send OTP

```
package com.javatechie.solid.srp;


public class BankService {

    public long deposit(long amount, String accountNo) {
        //deposit amount
        return 0;
    }

    public long withDraw(long amount, String accountNo) {
        //withdraw amount
        return 0;
    }
```

```
    public void printPassbook() {
        //update transaction info in passbook
    }

    public void getLoanInterestInfo(String loanType) {
        if (loanType.equals("homeLoan")) {
            //do some job
        }
        if (loanType.equals("personalLoan")) {
            //do some job
        }
        if (loanType.equals("car")) {
            //do some job
        }
    }

    public void sendOTP(String medium) {
        if (medium.equals("email")) {
            //write email related logic
            //use JavaMailSenderAPI
        }
    }

}
```

Have you imagined the scenario? Here the class has multiple reasons to change.

For example look into getLoanInterestInfo() method , now bank service provide only info for Personal Loan , Home Loan and car loan let's say in future bank people want to provide some other loan feature like **gold** loan and **study** loan then again you need to modify this class implementation right ?

similarly you can consider sendOTP() method , let's assume BankService support send OTP medium as a email but in future they

might want to introduced send OTP medium as Phone then again you need to change its implementation

it doesn't follow single responsibility principle because this class has to many responsible or task to perform

To achieve the goal of the single responsibility principle, we should implement a separate class that performs a single functionality only.

For Example , we can move Print related code snippet to Printer Service

```
public class PrinterService{
 public void printPassbook() {
        //update transaction info in passbook
    }
}
```

Similarly Loan related job

```
public class LoanService{
public void getLoanInterestInfo(String loanType) {
        if (loanType.equals("homeLoan")) {
            //do some job
        }
        if (loanType.equals("personalLoan")) {
            //do some job
        }
        if (loanType.equals("car")) {
            //do some job
        }
    }
}
```

similarly OTP related Job

```
public class NotificationService{public void sendOTP(String medium)
{
        if (medium.equals("email")) {
            //write email related logic
            //use JavaMailSenderAPI
        }
    }
}
```

Now if you observe Each class have single Responsibility to perform their task .which is exactly SRP says ...

Open closed Principle (OSP)

This principle states that "*software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification*" which means you should be able to extend a class behavior, without modifying it.

let's understand this principle with an example .let's consider the same Notification service which we just created .

```
public class NotificationService{public void sendOTP(String medium)
{
        if (medium.equals("email")) {
            //write email related logic
            //use JavaMailSenderAPI
        }
```

```
    }
}
```

Here as discussed earlier if you want to introduced send OTP via mobile Phone or WhatsApp number then you need to modify source code in Notification Service right ?

Here What OCP says , It open for Extension but close for modification hence its not recommended to modify Notification Service for each OTP Feature , it will violate OCP

So to overcome this you need to design your code in such a way that everyone can reuse your feature by just extending it and if they need any customization they can extend it and add their feature on top of it like a abstraction .

You can design something like below:

```
public interface NotificationService
{
public void sendOTP(String medium);
public void sendTransactionNotification(String medium);
}
```

EmailNotification implantation:

```
public class EmailNotification implements NotificationService{
public void sendOTP(String medium){
// write Logic using JavaEmail api
```

```
}
public void sendTransactionNotification(String medium){
}
}
```

Mobile Notification implementation:

```
public class MobileNotification implements NotificationService{
public void sendOTP(String medium){
// write Logic using Twilio SMS APIs
}
public void sendTransactionNotification(String medium){
}
}
```

similarly you can add implementation for WhatsApp notification service

```
public class WhatsAppNotification implements NotificationService{
public void sendOTP(String medium){
// write Logic using whatsapp API
}
public void sendTransactionNotification(String medium){
}
}
```

this approach would not affect the existing application. which follow OCP

Liskov substitution Principle (LSP)

This principle states that "*Derived or child classes must be substitutable for their base or parent classes*". In other words, if class

A is a subtype of class B, then we should be able to replace B with A without interrupting the behavior of the program.

This principle is bit tricky and interesting all it designed based on Inheritance concepts ,so let's better understand this with an example

Let's consider I have an abstract class called SocialMedia , who supported all social media activity for user to entertain them like below

```
package com.javatechie.solid.lsp;

public abstract class SocialMedia {

    public abstract  void chatWithFriend();

    public abstract void publishPost(Object post);

    public abstract  void sendPhotosAndVideos();

    public abstract  void groupVideoCall(String... users);
}
```

Social media can have multiple implantation or can have multiple child like Facebook, WhatsApp ,instagram and Twitter etc..

now let's assume Facebook want to use this features or functionality .

```
package com.javatechie.solid.lsp;

public class Facebook extends SocialMedia {

    public void chatWithFriend() {
```

```
        //logic
    }

    public void publishPost(Object post) {
        //logic
    }

    public void sendPhotosAndVideos() {
        //logic
    }

    public void groupVideoCall(String... users) {
        //logic
    }
}
```

In 20th century I believe everyone using this Facebook APP and all the above mention features available in Facebook so here we can consider Facebook is complete substitute of SocialMedia class , both can be replaced without any interrupt .

Now let's discuss WhatsApp class

```
package com.javatechie.solid.lsp;

public class WhatsApp extends SocialMedia {
    public void chatWithFriend() {
        //logic
    }

    public void publishPost(Object post) {
      //Not Applicable
    }

    public void sendPhotosAndVideos() {
      //logic
    }

    public void groupVideoCall(String... users) {
```

```
        //logic
    }
}
```

due to publishPost() method whatsapp child is not substitute of parents SocialMedia

because whatsapp doesn't support upload photos and videos for friend it's just a chatting application so it doesn't follow **LSP**

Similarly instagram doesn't support groupVideoCall() feature so we say instagram child is not substitute of parents SocialMedia

How to overcome this issue so that my code can follow **LSP**

solution

create a Social media interface

```
public interface SocialMedia {
   public void chatWithFriend();
   public void sendPhotosAndVideos()
}
public interface SocialPostAndMediaManager {
    public void publishPost(Object post);
}
public interface VideoCallManager{
   public void groupVideoCall(String... users);
}
```

Now if you observe we segregate specific functionality to separate class to follow **LSP**

now its up to implementation class decision to support features , based on their desired feature they can use respective interface for example instagram doesn't support video call feature so instagram implementation can be design something like this

```
public class Instagram implements SocialMedia
,SocialPostAndMediaManager{public void chatWithFriend(){
    //logic
   }
   public void sendPhotosAndVideos(){
    //logic
   }
   public void publishPost(Object post){
    //logic
   }}
```

This is how you can design **LSP**

Interface Segregation Principle (ISP)

This principle is the first principle that applies to Interfaces instead of classes in SOLID and it is similar to the single responsibility principle. It states that "*do not force any client to implement an interface which is irrelevant to them*".

For example let's say you have an interface called **UPIPayment** like below

```
public interface UPIPayments {

    public void payMoney();

    public void getScratchCard();

    public void getCashBackAsCreditBalance();
}
```

Now let's talk about few implementation for UPIPayments like Google Pay and Paytm

Google Pay support these features so he can directly implement this UPIPayments but Paytm doesn't support getCashBackAsCreditBalance() feature so here we shouldn't force client paytm to override this method by implementating UPIPayments .

we need to segregate interface based on client need , so to support this **ISP** we can design something like below

create a separate interface who will deal with Cashback

```
public interface CashbackManager{
 public void getCashBackAsCreditBalance();
}
```

Now we can remove getCashBackAsCreditBalance from UPIPayments interface .

Based on client need we segregate interface , let's say paytm now implements from UPIPayments then as a client we are not forcing him anything to use . which follow **ISP**

Dependency Inversion Principle (DIP)

The principle states that we must use abstraction (abstract classes and interfaces) instead of concrete implementations. High-level modules should not depend on the low-level module but both should depend on the abstraction

let's consider an best use case

You go to a local store to buy something, and you decide to pay for it by using your card. So, when you give your card to the clerk for making the payment, the clerk doesn't bother to check what kind of card you have given.

Even if you have given a debit card or credit card it not even matter; they will simply swipe it. this is what the abstraction between clerk and you to relay on Card processing

now let's replace this example in code to understand it better .

let's assume you have two option to do payments **Debit card** and **Credit card**

```
public class DebitCard{
public void doTransaction(int amount){
        System.out.println("tx done with DebitCard");
    }
}
```

Credit Card

```
public class CreditCard{
public void doTransaction(int amount){
        System.out.println("tx done with CreditCard");
    }
}
```

Now with this two card you went to shopping mall and purchased some order and decided to pay using CreditCard

```
public class ShoppingMall {private DebitCard debitCard;public
ShoppingMall(DebitCard debitCard) {
        this.debitCard = debitCard;
   }
public void doPayment(Object order, int amount){
debitCard.doTransaction(amount);
 }public static void main(String[] args) {
     DebitCard debitCard=new DebitCard();
     ShoppingMall shoppingMall=new ShoppingMall(debitCard);
     shoppingMall.doPayment("some order",5000);
    }
}
```

if you observe this is wrong design of coding , now ShoppingMall class tightly coupled with DebitCard

now there is some error in your debit card and user want to go with Credit card then this won't be possible because ShoppingMall is tightly couple with Debit Card

we can do that , remove Debit card from constructor and inject CreditCard. which not good approach to write code
because to follow **DIP** we need to design our application in such a way so that my shopping mall payment system should accept any type of ATM Card (it shouldn't care whether it is debit or credit card)

To simplify this designing principle further i am creating a interface called Bankcards like bellow

```
public interface BankCard {
  public void doTransaction(int amount);
}
```

Now both DebitCard and CreditCard will use This BankCard as abstraction

```
public class CreditCard implements BankCard{
public void doTransaction(int amount){
System.out.println("tx done with CreditCard");
    }
}
```

similarly DebitCard

```
public class DebitCard implements BankCard{
public void doTransaction(int amount){
System.out.println("tx done with DebitCard");
    }
}
```

Now you need to redesign Shopping mall implementation

```
public class ShoppingMall {private BankCard bankCard;public
ShoppingMall(BankCard bankCard) {
        this.bankCard = bankCard;
    }public void doPayment(Object order, int amount){
        bankCard.doTransaction(amount);
    }public static void main(String[] args) {
        BankCard bankCard=new CreditCard();
        ShoppingMall shoppingMall1=new ShoppingMall(bankCard);
        shoppingMall1.doPayment("do some order", 10000);
    }
}
```

Now if you observe shopping mall is loosely coupled with BankCard , any type of card process the payment without any impact . which proofs **DIP**

PrincipleDescription

Single Responsibility Principle : Each class should be responsible for a single part or functionality of the system.

Open-Closed Principle : Software components should be open for extension, but not for modification.**Liskov Substitution**

PrincipleObjects of a superclass should be replaceable with objects of its subclasses without breaking the system.**Interface Segregation Principle**No client should be forced to depend on methods that it does not use.**Dependency Inversion Principle**High-level modules should not depend on low-level modules, both should depend on abstractions.

Hope this SOLID principle concepts clear for you . if you like it do let me show your interest in comment section .

YouTube : https://youtu.be/BM_lSZPMClo

SOLID Principles in 10 minutes

Understand SOLID Principles in less than a hour

In this article will focus to discuss about explanation of *SOLID Principle* and give same example (violation and compliance) in java code. SOLID principles helps us to make our software more readable, understandable, flexible and maintainable.

SOLID Principle is just acronym of five principles.
1) S — Single Responsibility Principle
2) O — Open/Closed Principle
3) L — Liskov's Substitution Principle

4) I — Interface Segregation Principle

5) D — Dependency Inversion Principle

S — Single Responsibility Principle (SRP)

A class should have only a single responsibility and we only have one reason to change or modify it's class. If we have another reason to change class then we need to split into different class.

```
public
class
Service
{
            public void insertStudent(Student student){
                // insert student
            }
            public void updateStudent(Student student){
                // update student
            }
            public void deleteStudent(Student student){
                // delete student
            }
            public void insertCourse(Course course){
                // insert course
            }
            public void updateCourse(Course course){
                // update course
            }
            public void deleteCourse(Course course){
                // delete course
            }
        }
```

In code above shows violation of SRP because *Service.java* handle the business logic for Student and Course class. In the future *Service.java* can become bloaters or even become our 'nightmare' because we put *everything* in *Service.java*.

The solution is we need to split *Service.java* into specific class to handle business logic for Student (*StudentService.java*) and Course (*CourseService.java*).

```
public class
CourseService
{
                public void insert(Course course){
                    // insert course
                }
                public void update(Course course){
                    // update course
                }
                public void delete(Course course){
                    // delete course
                }
            }
            ...
            public class StudentService {
                public void insert(Student student){
                    // insert student
                }
                public void update(Student student){
                    // update student
                }
                public void delete(Student student){
                    // delete student
                }
            }
```

O — Open/Closed Principle

Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification. It's means the entity that we create should be extendable. By implementing this principle we are able to create a new feature/ behaviour without breaking the existing code.

```
public
class
Student
{
            private String type;

            public void introduce() {
                if ("highSchool".equals(type)) {
                    System.out.println("I am an high school student");
                } else if ("underGraduate".equals(type)) {
                    System.out.println("I am a under graduate student");
                } else {
                    System.out.println("I am a common student");
                }
            }
        }
```

The violation of Open Closed Principles

The violation is when we need to add a new type of Student, we need to modify our code by adding a new codition. Let's imagine that *Student.java* is already compiled and we use *Student.java* as library in our code, then until some day we need to add new type of Student. We can not do anything (or do everything in bad way) until the author of *Student.java* patch their code by adding a new condition.

For this case we can use *inheritance* to solve the problem.

```
public
class
Student
{
            public void introduce() {
                System.out.println("I am a common student");
            }
        }
        ...
        public class HighSchoolStudent extends Student {
            @Override
            public void introduce() {
                System.out.println("I am an high school student");
            }
        }
        ...
        public class UnderGraduateStudent extends Student {
            @Override
            public void introduce() {
                System.out.println("I am a under graduate student");
            }
        }
```

The solution of violation

L — Liskov's Substitution Principle (LSP)

"Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.".

Parent class should extendable by subclass without blowing up the software

I — Interface Segregation Principle (ISP)

Many client-specific interfaces are better than one general-purpose interface. Here, the idea is split the 'big' interface to the smaller interface until client of the interface will only know about the methods that are related to them.

```
public
interface
Study {
            void createPaper();
            void createThesis();
        }
        ...
        public class Student implements Study {
            public void createPaper() {
                // create a paper
            }
            public void createThesis() {
                throw new IllegalStateException("not implement yet");
            }
        }
        ...
        public class UnderGraduateStudent extends Student {
            @Override
            public void createThesis() {
                // create a thesis
            }
        }
```

The violation is implementation of Study interface in *Student.java* because by using *Study* interface in *Student.java*, *Student.java* forced to implement two methods; *createPaper()* and *createThesis()*. Actually, there is no logic for *createThesis()* in *Student.java*.

We need to split the general interface, in the case is Study interface, to more specific interface.

```
public
interface
PaperAble
{
            void createPaper();
        }
        ...
        public interface ThesisAble {
            void createThesis();
        }
        ...
        public class Student implements PaperAble {
            public void createPaper() {
                // create a paper
            }
        }
        ...
        public class UnderGraduateStudent extends Student implements ThesisAble {
            public void createThesis() {
                // create a thesis
            }
        }
```

D — Dependency Inversion Principle

Dependency Inversion is a great solution to reduce tight coupling in our software. Depend upon abstractions, not concretions.

> **“High-level modules should not depend on low level modules, rather both should depend on abstraction. Abstraction should not depend on details; rather detail should depend on abstraction.”**

let’s straight to example :D

```
public
class
Main {
         public static void main(String[] args) {
            MySqlConnection connection = new MySqlConnection();
```

```
        StudentRepository studentRepository = new StudentRepository(connection);
        studentRepository.save();
        studentRepository.delete();
        TeacherRepository teacherRepository = new TeacherRepository(connection);
        teacherRepository.save();
        teacherRepository.update();
    }
}
...
public class StudentRepository {
    private MySqlConnection connection;
    public StudentRepository(MySqlConnection connection){
        this.connection = connection;
    }
    public void save() {
        connection.insert();
    }
    public void delete() {
        connection.delete();
    }
}
...
public class TeacherRepository {
    private MySqlConnection connection;
    public TeacherRepository(MySqlConnection connection){
        this.connection = connection;
    }
    public void save() {
        connection.insert();
    }
    public void update() {
        connection.update();
    }
}
```

The violation is in repository class because that classes are depend on *MySqlConnection.java* to finish their method (insert, update,

delete). Now, what happend if we need to change *MySqlConnection.java* to *PsqlConnection.java* ? We need modify all of classes that depend on *MySqlConnection.java*, it's will be very dangerous because we need to modify a lot of class.

To solve this case, we can create an abstraction class that will implement in *MySqlConnection.java* and use that abstraction in our repository class.

```
public
class
Main {
          public static void main(String[] args) {
      //        Connection connection = new MySqlConnection();
              Connection connection = new PsqlConnection( );
              StudentRepository studentRepository = new StudentRepository(connection);
              studentRepository.save();
              studentRepository.delete();
              TeacherRepository teacherRepository = new TeacherRepository(connection);
              teacherRepository.save();
              teacherRepository.update();
          }
      }
      ...
      public interface Connection {
          void insert();
          void update();
          void delete();
      }
      ...
      public class MySqlConnection implements Connection {
          public void insert() { }
          public void update() { }
          public void delete() { }
      }
      ...
      public class StudentRepository {
          private Connection connection;
          public StudentRepository(Connection connection) {
```

```
            this.connection = connection;
        }
        public void save() {
            this.connection.insert();
        }
        public void delete() {
            this.connection.delete();
        }
    }
```

Important Java Design Patterns You Need to Know About

Design patterns are general solutions to routine problems in software design. Every **pattern** acts as a **blueprint** that allows customization to solve a given design problem in any code during the development of software modules. I have lined up the docket for this article as below:

- Why do we need Design Patterns?
- What are Design Patterns?
- Structure of a Design Pattern

- Types of Design Patterns
- Creational Design Pattern
- Structural Design Pattern
- Behavioural Design Pattern
- JEE Design Pattern
- Overview of design patterns

Why do we need Design Patterns?

A **design pattern** systematically names, motivates and explains a general design that addresses a recurring design problem in object-oriented systems. **Design patterns** are needed to represent some of the best practices followed and adopted in software development.

What are Design Patterns?

Software Design Pattern can be defined as a **software template** or a description to solve a problem that occurs in multiple instances while designing a Software Application or a Software Framework.

The credit of Design Patterns goes to the **Gang of Four.** *Erich Gamma, Richard Helm, Ralph Johnson* and *John Vlissides* were the authors of the book on the Java Design Patterns.

Structure of a Design Pattern

Structure of any Design Pattern is considered to be highly **organised.** They are often documented in the form of a **template** such that, the users can **visually identify** the problem and instantaneously find the solution to it, based on the **relationship** between **classes** and **objects.** The Design Patterns Template described by the original authors of the Design Patterns is as follows:

Term	Description
Pattern Name	Describes the essence of the pattern in a short, but expressive name.
Intent	Describes what the Pattern Does
Also, known as	List any synonyms of the pattern
Motivation	Provides an example of the problem and describes the solution to it
Applicability	Lists the situations where the pattern is applicable
Structure	Set of diagrams of the classes and objects that depict the pattern
Participants	Describes the classes and objects that participate in the design
Collaborations	Describes the Participants collaboration and their responsibilities.
Consequences	Describes the forces that exist with the pattern and the benefits

Types of Design Patterns

Typically, Java Design Patterns are divided into **Four Categories** and each of those are further classified as below:



- **Creational Design Patterns** are concerned with the method of creating Objects.
- **Structural Design Patterns** deal with the composition of classes and objects which form larger structures.
- **Behaviour Design Patterns** are concerned with the responsibility and interaction between the objects.
- **JEE Design Patterns** are concerned with providing solutions to the Java EE-based software applications and frameworks.

Creational Design Patterns

are preferred during the process of class initiation. The Creational Design Patterns are further classified as follows:

1. **Factory Pattern**
2. Abstract Factory Pattern
3. **Singleton Pattern**
4. Prototype Pattern
5. Builder Pattern.
6. Object Pool Pattern

Let us discuss some important Creational Java Design Patterns practically.

Factory Design Pattern: It follows the principle of "*Define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate*"**.** The Factory Method Pattern is also known as **Virtual Constructor.**

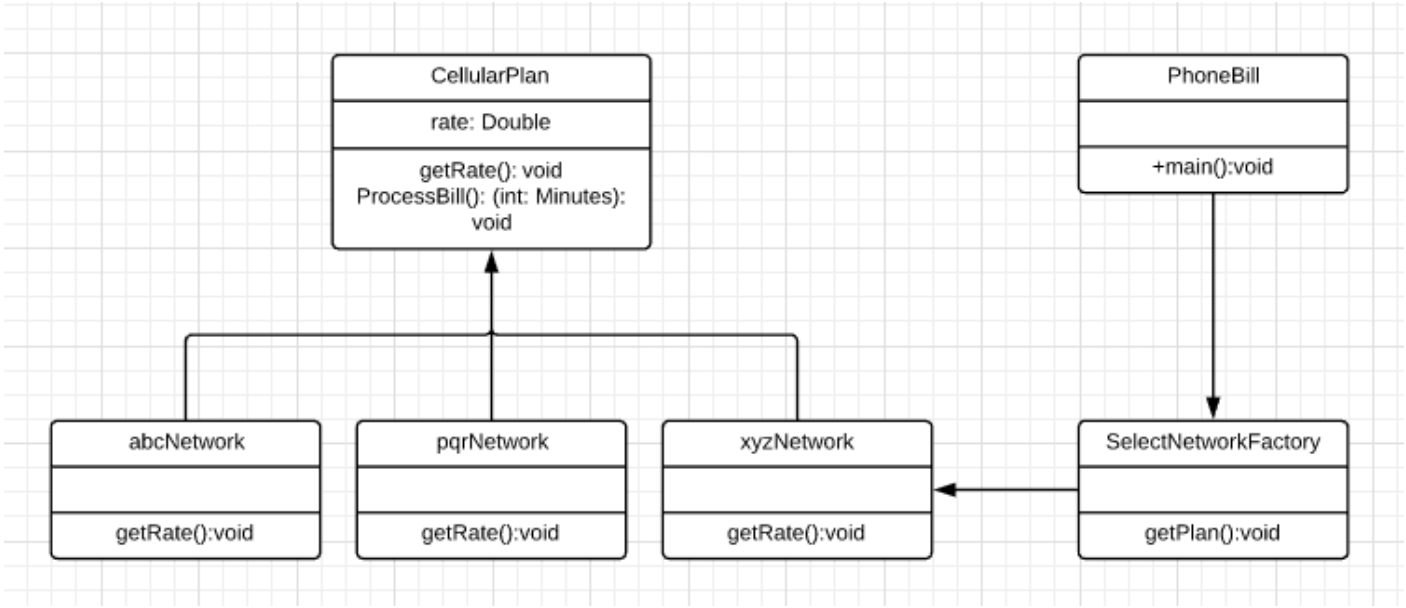
Advantages:

- It is designed to allow all the sub-classes to choose the type of objects so as to create them.

Example: We have three **Cellular Network Plan** which describes the call cost per minute. Here we have three different networks namely, **abcNetwork, pqrNetwork,** and **xyzNetwork** along with their *charges per minute*. Let us find out the cost of a certain number of minutes on each network.



UML Diagram:



//cellular plan

//abcNetwork

//pqrNetwork

//xyzNetwork

//SelectNetworkFactory

//PhoneBill

Output:

```
Enter the name of network for which the bill will be generated:
abcNetwork
```

```
Enter the number of minutes for bill will be calculated: 35
Bill amount for abcNetwork of 35 units is: 52.5
```

Singleton Design Pattern: It follows *"define a class that has only one instance and provides a global point of access to it*". The class must ensure that only a **single instance** should be created and a **single object** can be used by all other classes.

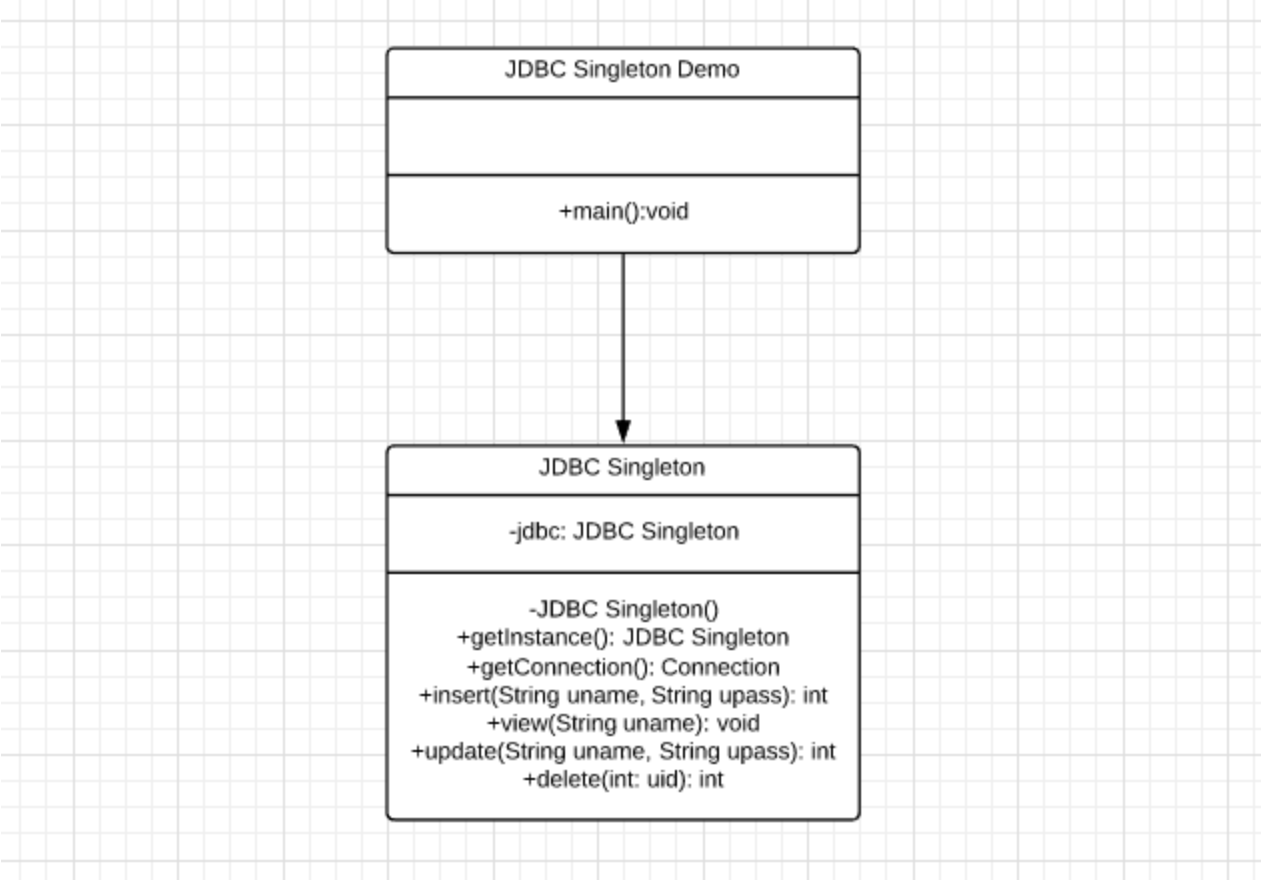
Advantages:

- It is designed to **save memory.**
- The object is not created on every request, instead, a **single instance** is reused.

Example: We have a **MySQL** Database. Let us insert data into the database using one single object instance. Here, we have 4 different operations that are performed onto the database and all of those operations use one **single object instance.**



UML Diagram:



Output:

```
DATABASE OPERATIONS

---------------------

1. Insertion

2. View

3. Delete

4. Update

5. Exit


Please enter the choice what you want to perform in the database: 1

Enter the username you want to insert data into the database: Ravi
```

```
Enter the password you want to insert data into the database: 1234
Enter the ID you want to insert data into the database: 1011
1 Data has been inserted successfully
Press Enter key to continue...
```

Structural Design Patterns

deal with the **composition** of classes and objects which form larger structures. They simplify the structure by identifying the **relationship** between the **classes** and **objects.** The Structural Design Patterns are further classified as follows:

1. **Facade Pattern**
2. Bridge Pattern
3. Composite Pattern
4. Decorator Pattern
5. **Adapter Pattern**
6. Flyweight Pattern
7. Proxy Pattern
8. Filter Pattern

Let us discuss some important Structural Java Design Patterns practically.

Facade Design Pattern: Describes a **higher-level** interface that makes the subsystem easier to use. Every **Abstract Factory** is a Facade Design Pattern.

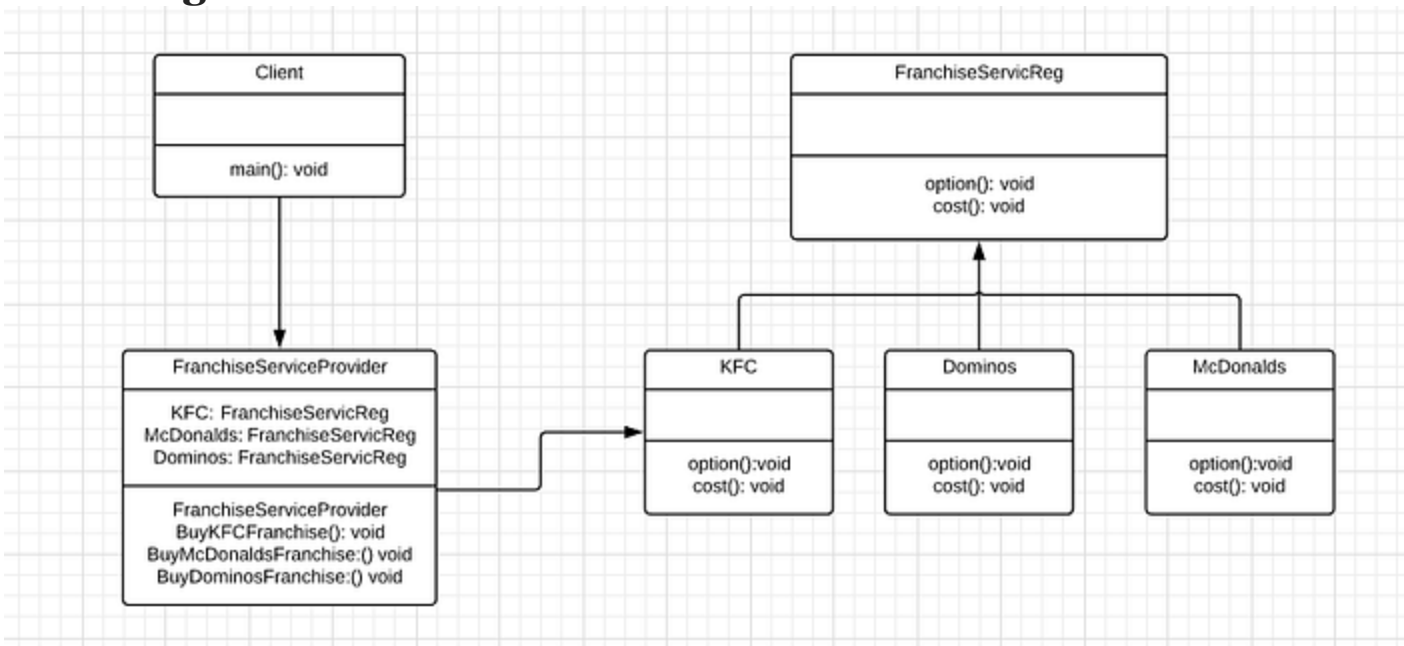
Advantages:

- It protects the end-users from the complex sub-system components.

Example: Now, let us use the **Facade Design pattern** to find out the cost of the franchise you wish to buy.



UML Diagram:



Output:

```
Welcome to Franchise Service Registration...!
1. KFC
2. McDonalds
3. Dominos
4. EXIT
Please Enter your Franchise Option Number: 1
KFC
Rs 1,00,00,000
```

Adapter Design Pattern: Provides the **interface** according to client requirement while using the services of a class with a **different interface.** The Adapter Pattern is also known as **Wrapper.**

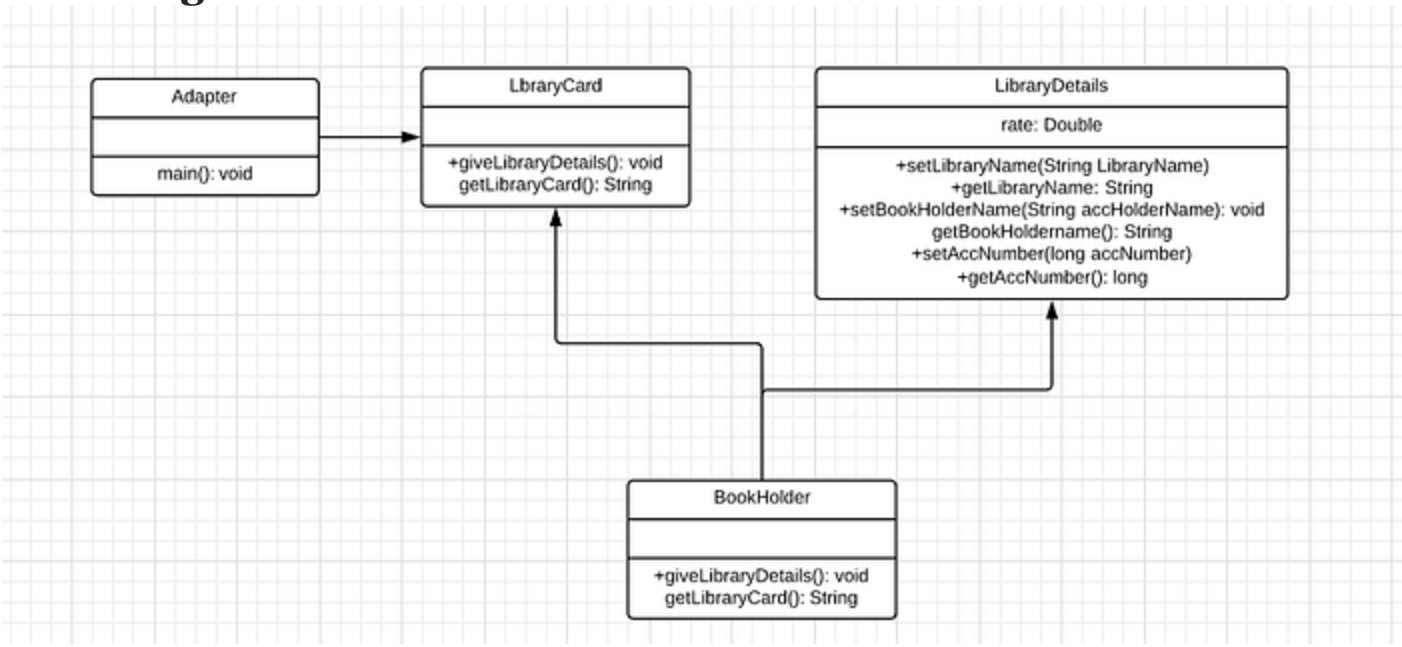
Advantages:

- It is designed to enable two or more previously incompatible objects to **interact** with each other.

Example: Here is a simple **Library Card** example which is designed to issue a **Library card** to a new user of the library which includes all the details like a *book holder ID, Account numbe*r and many more. Let us execute this using **Adapter Design Pattern.**



UML Diagram:



Output:

```
Enter the Library account holder name :Ravi

Enter the account number of the library:12012

Enter the Library name :MG Library
The Account number 12012 of Ravi in MG Library Library is valid and
authenticated for issuing the Library card.
```

Behavioural Design Patterns

are concerned with the responsibility and **interaction** between the objects. These Design Patterns make sure the objects are **loosely coupled,** yet, can easily communicate with each other. Behavioural Design Patterns are further classified as follows:

1. **Chain Of Responsibility Pattern**



1. **Strategy Pattern**
2. Interpreter Pattern
3. Iterator Pattern
4. Mediator Pattern
5. Memento Pattern
6. **Command Pattern**

7. State Pattern
8. **Observer Pattern**
9. Template Pattern
10. Visitor Pattern

Let us discuss some important Behavioural Design Patterns practically.

Strategy Design Pattern: It defines a family of **functionality** and **encapsulates** each one, and make them **interchangeable.** The Strategy Pattern is also known as **Policy.**

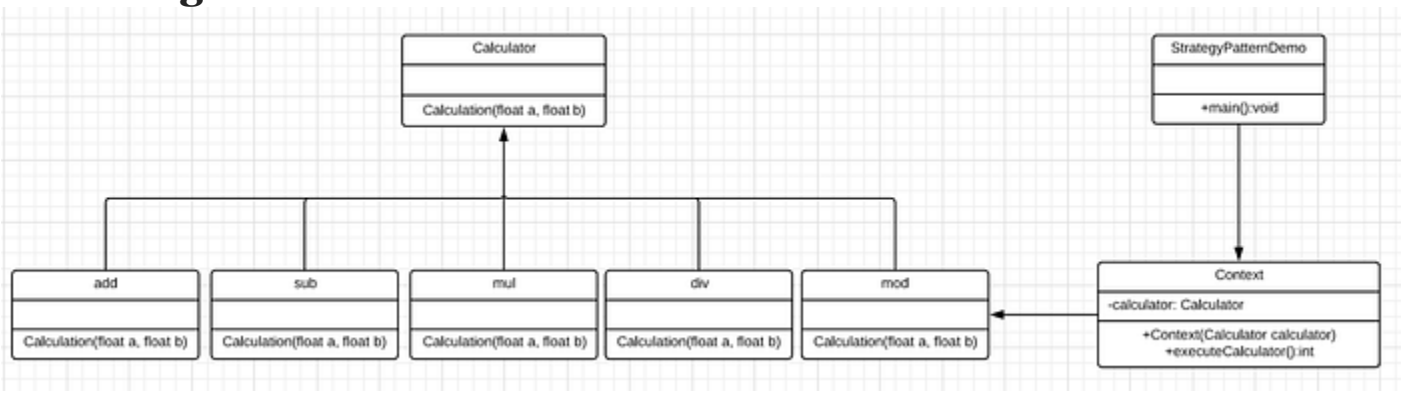
Advantages:

- It is designed to provide a substitute for sub-classes.

Example: we are going to consider a simple example of a **calculator.** We need to perform five different operations. We shall encapsulate all those five operations into Calculator class and execute the program using the **Strategy Design Pattern.**



UML Diagram:



Output:

```
Enter the first value: 10

Enter the second value: 2

Addition = 12.0

Subtraction = 8.0

Multiplication = 20.0

Multiplication = 5.0

Multiplication = 0.0
```

Command Design Pattern: It **encapsulates** a request under an object as a command and passes it to the **invoker object.** Invoker object looks for the appropriate object which can handle this command and pass the command to the corresponding object and that object executes the command. It is also known as **Action** or **Transaction.**

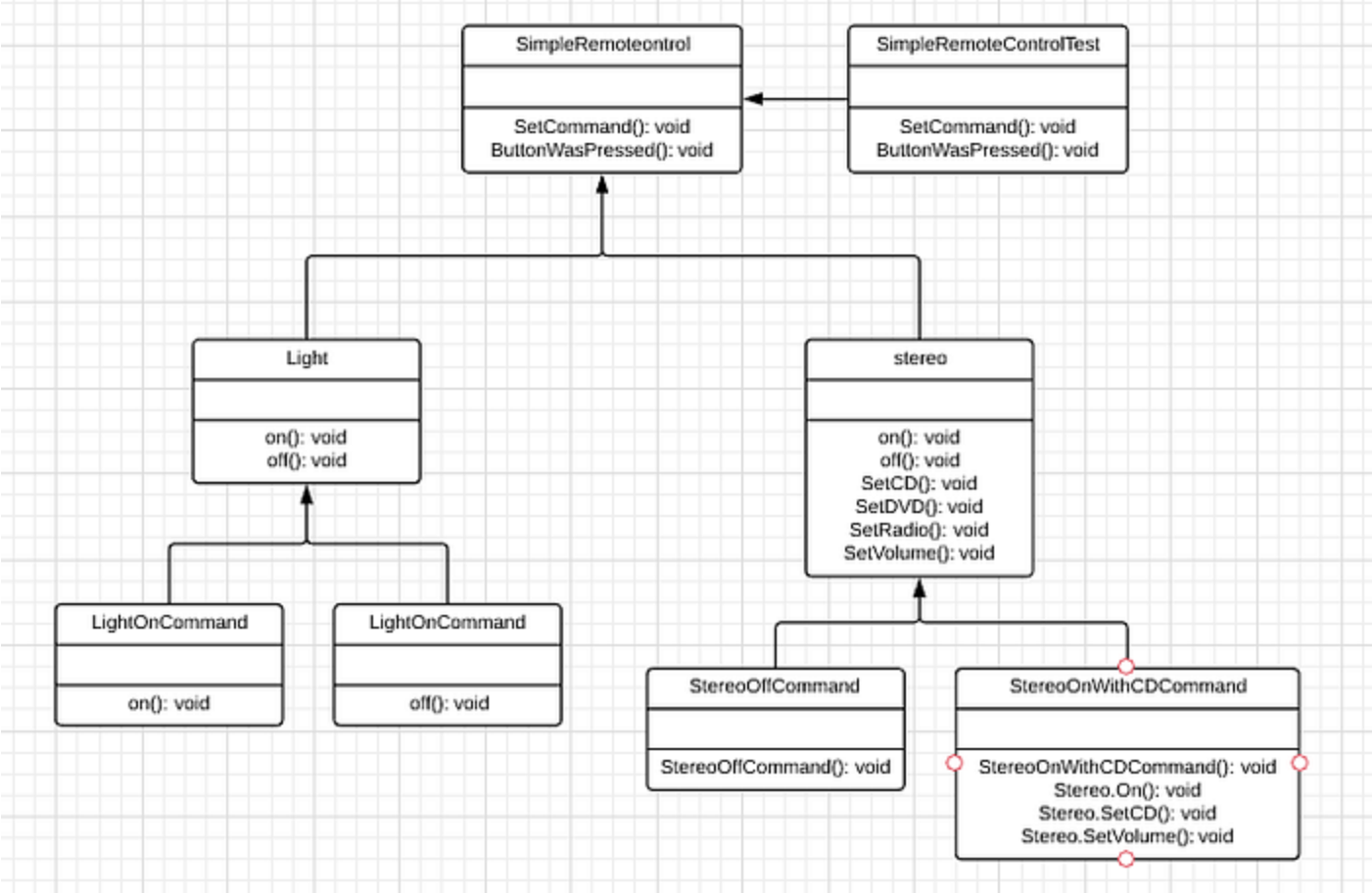
Advantages:

- It separates the object that **invokes** the operation with the object that **actually performs the operation.**

Example: This example demonstrates a simple command execution cycle where the user requires to exhibit **switching on** and **off** the various electronic devices in his houses like a **bulb** and **stereo player.** He invokes the command through an **invoker** object called a **simple remote control.**



UML Diagram:



Output:

```
Light is on

Stereo is on

Stereo is set for CD input

Stereo volume set to 11

Stereo is off
```

Observer Design Pattern: It defines a **one-to-one** dependency so that when one object changes state, all its dependents are notified and

updated automatically. The Memento pattern is also known as **Dependents** or **Publish-Subscribe.**

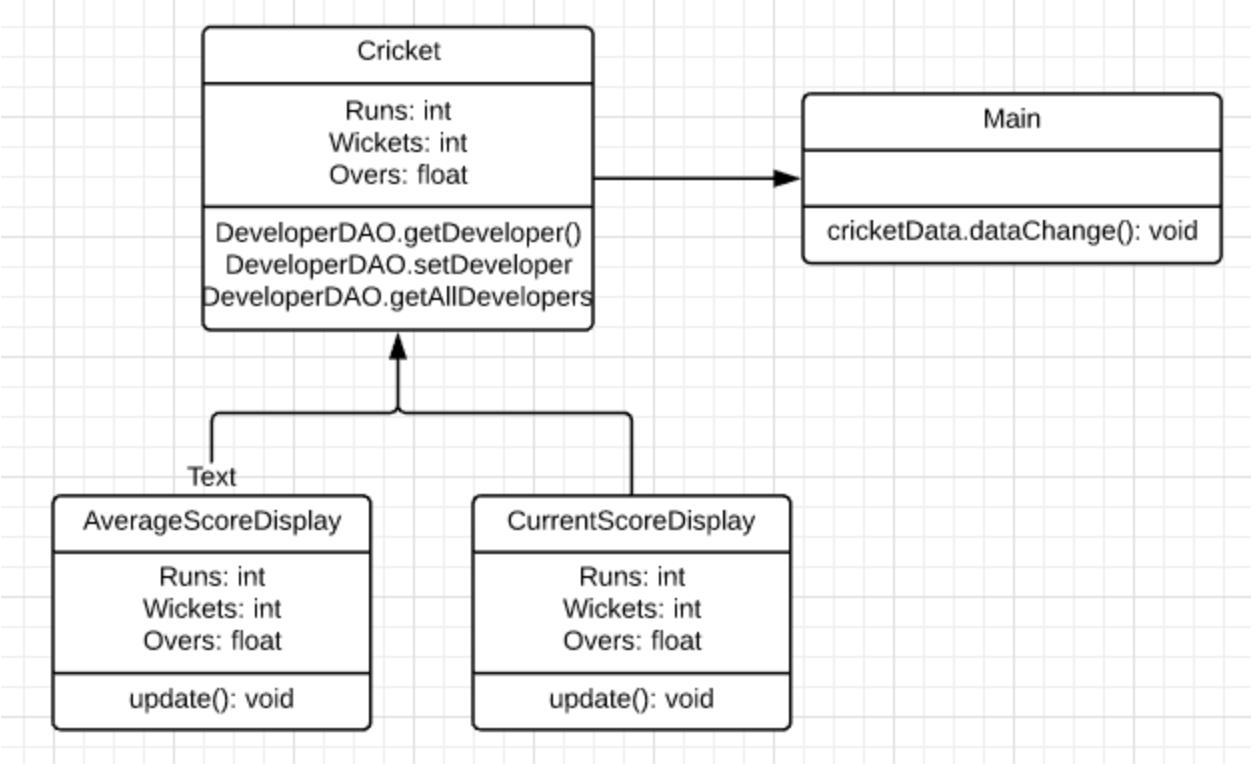
Advantages:

- It illustrates the coupling between the **objects** and the **observer.**
- It provides support for **broadcast-type** communication.

Example: We are going to execute a program using the **Observer** Design Pattern to display the **current average** score and the **current predictable** score of a cricket match.



UML Diagram:



Output:

```
Current Score Display:

Current Runs: 90

Wickets Lost:2

Overs Played: 10.2


Average Score Display:

Average Run Rate: 8.823529

Predictable Score: 441
```

JEE Design Patterns

are concerned with providing solutions to the **Java EE-based** software applications and frameworks. These patterns are widely used in **Spring.** JEE Design Patterns are further classified as follows:

1. **MVC Design Pattern**
2. Dependency Injection Pattern
3. **DAO Design Pattern**
4. Business Delegate Pattern
5. Intercepting Filter Pattern
6. Service Locator Pattern
7. Transfer Object Pattern

Let us discuss some important JEE Design Patterns practically.

MVC Design Pattern: MVC Design Pattern is defined as follows:

- **Models** are basically objects used as blueprints for all of the objects that will be used in the application.
- **Views** are used to represent the presentational aspect of the information and data located in the models.

- **Controllers** control and act as both of the Models as well as Views. They serve as a connection between the Models and Views. Controllers are capable to instantiate, update and delete models. They load them with information and then send the data to the views to present to the end-user.

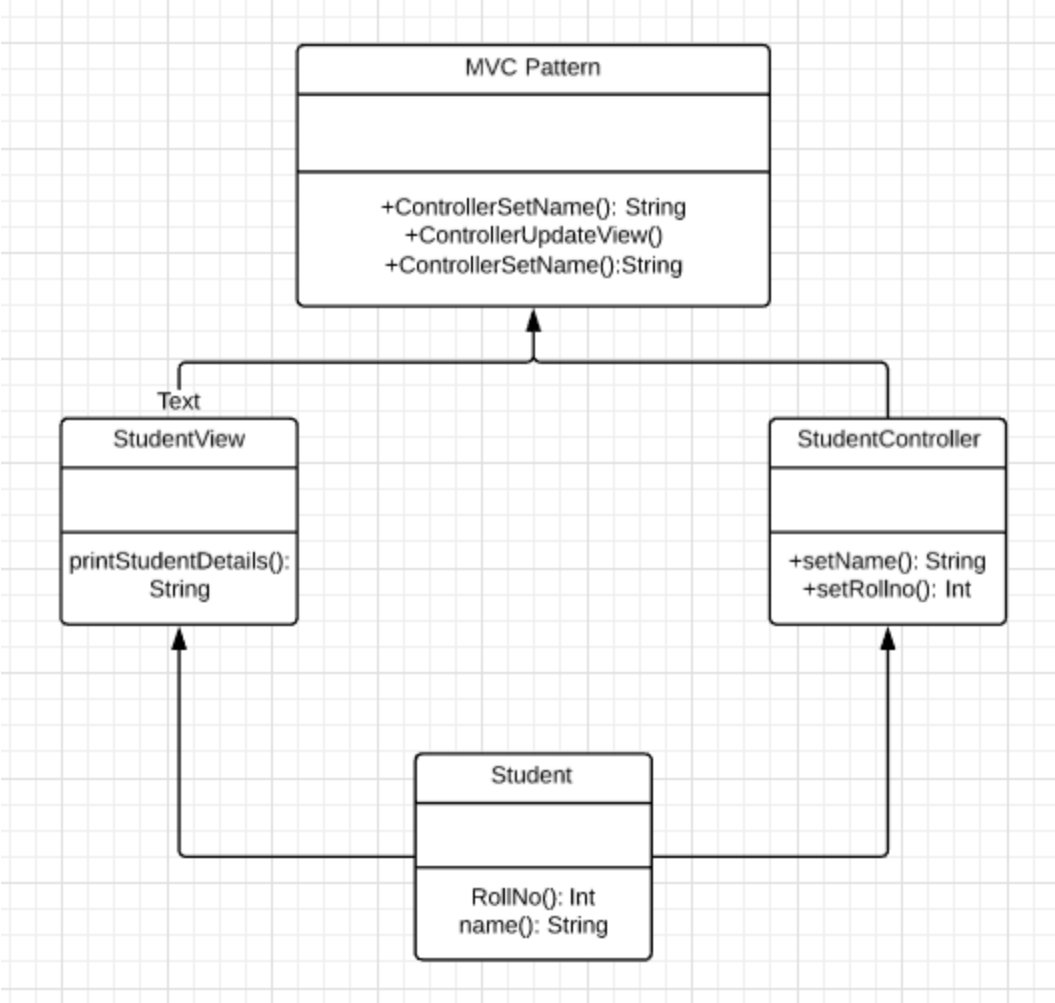
Advantages:

- It support **multiple views** for a model.

Example: We are going to use **MVC Design** Pattern to set and print the data of the school students.



UML Diagram:



Output:

```
Student:

Name: Karan Kumar

Roll No: 21CSE987

Student:

Name: Sandeep Shukla

Roll No: 21CSE987
```

DAO Design Pattern: DAO is a pattern in which objects are dedicated to the communication with the **Data Layer.** These objects instantiate **"SessionFactories"** and handle all of the logic behind communicating with the database.

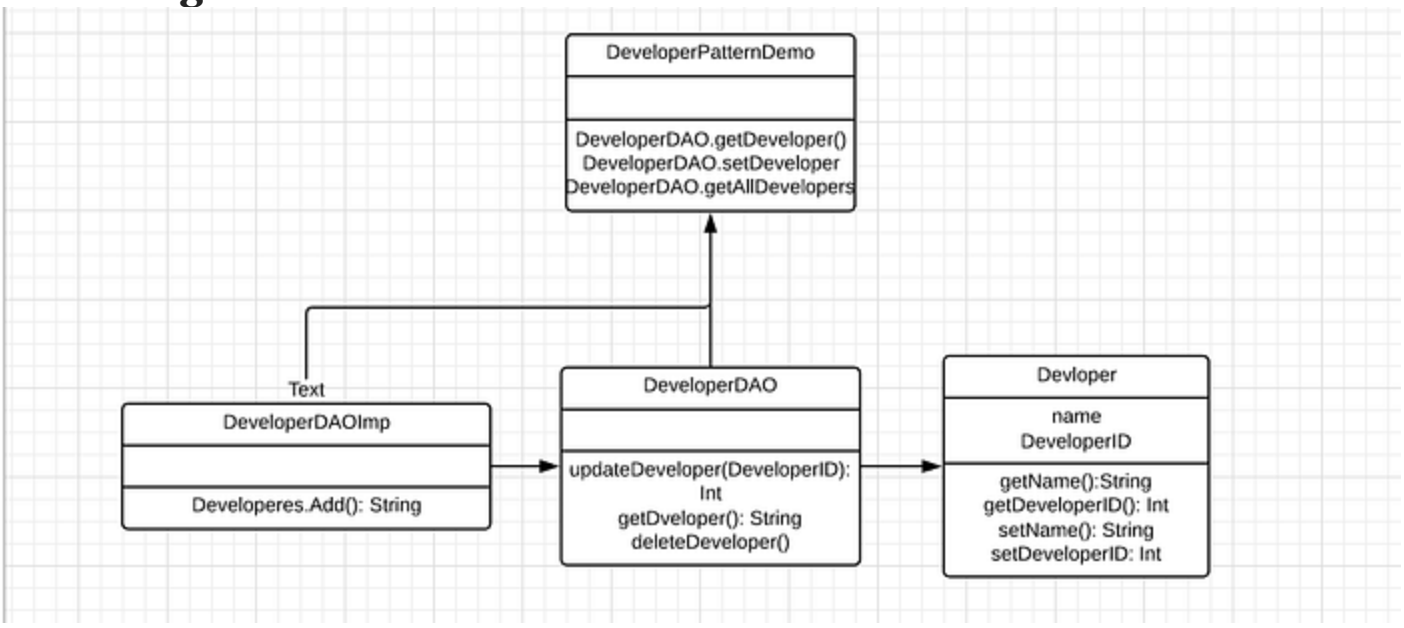
Advantages:

- It uses common calls to **retrieve objects.**

Example: We are going display the Developer details of a certain IT company using the **DAO Design Pattern.**



UML Diagram:



Output:

```
DeveloperId : 0, Name : Kushagra
DeveloperId : 1, Name : Vikram
DeveloperId 0, updated in the database
DeveloperId : 0, Name : Lokesh
```

Overview of Design Patterns

- Java Design Patterns are designed to be **reusable** in multiple projects.
- They furnish solutions that help to define **system architecture** with ease.
- Design Patterns are capable to capture the software engineering experiences.
- They provide **transparency** to the design of an application.
- Design Patterns are provided by **Expert Developers** which prove that these are testified and **well-proven** solutions.
- Java Design Patterns provide **clarity** to the **System Architecture** and provide the possibility of building a **better system.**